

Chapter 3. Compute eigenvalues and eigenvectors

ARIN7013 Numerical methods in artificial intelligence

The University of Hong Kong



Objective

Compute eigenvalues and eigenvectors (Rayleigh quotient, with applications in artificial intelligence and machine learning).

Table of Contents

1	Rayleigh quotient	4
2	Numerical schemes for finding eigenvalues and eigenvectors	20
	■ Power iteration	23
	■ Inverse iteration	31
	■ Rayleigh quotient iteration	38
3	Formulation using Lagrange multipliers	50
4	Principal Components Analysis (PCA)	56
	■ Motivation and main idea	57
	■ Interpretation of PCA as Variance Maximization	60
	■ Implementation of PCA	73
	■ Kernel PCA	83
5	Applications of PCA	94
	■ Application: US Zero-Coupon Rates	96
	■ Application: Netflix recommendation system (Self-study)	126

3.1 Rayleigh quotient

Recall

Denote by $\langle \cdot, \cdot \rangle$ the standard inner product on $\mathbb{R}^n$. The real $n \times n$ matrix $\mathbb{M}$ is symmetric if and only if

$$\langle \mathbb{M}\mathbf{x}, \mathbf{y} \rangle = \langle \mathbf{x}, \mathbb{M}\mathbf{y} \rangle \quad \forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^n.$$

Number the eigenvalues of $\mathbb{M}$ so that $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$. Denote the corresponding eigenvectors as $\{\psi_1, \psi_2, \dots, \psi_n\}$. By the definition, each eigenvalue and its corresponding eigenvector satisfy

$$\mathbb{M}\psi_i = \lambda_i\psi_i$$

Remark

For a symmetric matrix $\mathbb{M}$, $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$ are real numbers and $\{\psi_1, \psi_2, \dots, \psi_n\}$ form an orthogonal basis.

Orthogonal basis

To see orthogonality, suppose ψ_i and ψ_j are eigenvectors corresponding to distinct eigenvalues λ_i and λ_j . Then

$$\begin{aligned}\lambda_i \langle \psi_i, \psi_j \rangle &= \langle \lambda_i \psi_i, \psi_j \rangle \\ &= \langle \mathbb{M} \psi_i, \psi_j \rangle \\ &= \langle \psi_i, \mathbb{M} \psi_j \rangle \\ &= \langle \psi_i, \lambda_j \psi_j \rangle \\ &= \lambda_j \langle \psi_i, \psi_j \rangle \quad \text{for any } i \neq j\end{aligned}$$

Since λ_i and λ_j are distinct, we have $\langle \psi_i, \psi_j \rangle = 0$. Therefore, $\{\psi_1, \psi_2, \dots, \psi_n\}$ forms an orthogonal basis.

What is Rayleigh quotient?

Definition (Rayleigh quotient)

The Rayleigh quotient of a real vector $\mathbf{x}$ ($\mathbf{x} \neq \mathbf{0}$) with respect to a real symmetric matrix $\mathbb{M}$ is the ratio

$$R(\mathbb{M}, \mathbf{x}) := \frac{\mathbf{x}^T \mathbb{M} \mathbf{x}}{\mathbf{x}^T \mathbf{x}} = \frac{\langle \mathbb{M} \mathbf{x}, \mathbf{x} \rangle}{\langle \mathbf{x}, \mathbf{x} \rangle}$$

Note that:

- $R(\mathbb{M}, c\mathbf{x}) = R(\mathbb{M}, \mathbf{x})$ for any non-zero scalar c ;
- If $\boldsymbol{\psi}$ is an eigenvector of $\mathbb{M}$ with eigenvalue λ , then

$$\frac{\langle \mathbb{M} \boldsymbol{\psi}, \boldsymbol{\psi} \rangle}{\langle \boldsymbol{\psi}, \boldsymbol{\psi} \rangle} = \frac{\langle \lambda \boldsymbol{\psi}, \boldsymbol{\psi} \rangle}{\langle \boldsymbol{\psi}, \boldsymbol{\psi} \rangle} = \frac{\lambda \langle \boldsymbol{\psi}, \boldsymbol{\psi} \rangle}{\langle \boldsymbol{\psi}, \boldsymbol{\psi} \rangle} = \lambda.$$

Examples of Rayleigh quotient

Example

Consider the matrix

$$\mathbb{A} = \begin{bmatrix} 3 & 4 \\ 4 & 3 \end{bmatrix}$$

for which the exact eigenvalues are $\lambda_1 = -1$, $\lambda_2 = 7$, with corresponding eigenvectors $\mathbf{v}_1 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$, $\mathbf{v}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

$$R(\mathbb{A}, \mathbf{v}_1) = \frac{\mathbf{v}_1^T \mathbb{A} \mathbf{v}_1}{\mathbf{v}_1^T \mathbf{v}_1} = \frac{[-1 \ 1] \begin{bmatrix} 3 & 4 \\ 4 & 3 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix}}{2} = \frac{-2}{2} = -1$$

$$R(\mathbb{A}, \mathbf{v}_2) = \frac{\mathbf{v}_2^T \mathbb{A} \mathbf{v}_2}{\mathbf{v}_2^T \mathbf{v}_2} = \frac{[1 \ 1] \begin{bmatrix} 3 & 4 \\ 4 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix}}{2} = \frac{14}{2} = 7$$

Theorem

Let $\mathbb{M} \in \mathbb{R}^{n \times n}$ be a symmetric matrix and let

$$\mathbf{x} := \arg \max_{\substack{\mathbf{x} \in \mathbb{R}^n \\ \mathbf{x} \neq \mathbf{0}}} R(\mathbb{M}, \mathbf{x}).$$

Then

$$\mathbb{M}\mathbf{x} = R(\mathbb{M}, \mathbf{x})\mathbf{x}.$$

Moreover, this eigenvalue $R(\mathbb{M}, \mathbf{x}) = \sigma_n$ is the largest eigenvalue of $\mathbb{M}$.

Remark

Such a vector is guaranteed to exist. It suffices to consider vectors $\mathbf{x}$ of unit norm, which is a **compact set** (the concept of 'compact' is equivalent to 'close and bounded' in Euclidean space). As the Rayleigh quotient is a continuous function away from the origin, there exists a vector in this set at which it is maximized.

1/3.

Proof. Let $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$ be the eigenvalues of $\mathbb{M}$ and $\{\psi_1, \psi_2, \dots, \psi_n\}$ to be the corresponding eigenvectors. Normalize the eigenvectors to form an orthonormal basis $\{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n\}$. Consider decomposing an arbitrary vector $\mathbf{x}$ in this basis as

$$\mathbf{x} = \sum_{i=1}^n \alpha_i \mathbf{v}_i,$$

where

$$\alpha_i = \langle \mathbf{x}, \mathbf{v}_i \rangle$$

is the coordinate of $\mathbf{x}$ orthogonally projected onto $\mathbf{v}_i$. WLOG, we can assume that $\mathbf{x}$ is a unit vector. That is, $\langle \mathbf{x}, \mathbf{x} \rangle = \|\mathbf{x}\|^2 = 1$. We write this

in the eigenbasis as $\sum_{i=1}^n \langle \alpha_i \mathbf{v}_i, \alpha_i \mathbf{v}_i \rangle = 1$. □

2/3.

Evaluate the Rayleigh quotient with respect to $\mathbf{x}$:

$$\begin{aligned}
 \frac{\langle \mathbf{M}\mathbf{x}, \mathbf{x} \rangle}{\langle \mathbf{x}, \mathbf{x} \rangle} &= \langle \mathbf{M}\mathbf{x}, \mathbf{x} \rangle \\
 &= \left\langle \mathbf{M} \sum_{i=1}^n \alpha_i \mathbf{v}_i, \sum_{i=1}^n \alpha_i \mathbf{v}_i \right\rangle \\
 &= \left\langle \sum_{i=1}^n \alpha_i \mathbf{M} \mathbf{v}_i, \sum_{i=1}^n \alpha_i \mathbf{v}_i \right\rangle \\
 &= \left\langle \sum_{i=1}^n \alpha_i \lambda_i \mathbf{v}_i, \sum_{i=1}^n \alpha_i \mathbf{v}_i \right\rangle \\
 &= \left\langle \sum_{i=1}^n \langle \mathbf{x}, \mathbf{v}_i \rangle \lambda_i \mathbf{v}_i, \sum_{i=1}^n \langle \mathbf{x}, \mathbf{v}_i \rangle \mathbf{v}_i \right\rangle.
 \end{aligned}$$

3/3.

Since $\{\mathbf{v}_i : i = 1, \dots, n\}$ is a set of orthonormal basis (ONB), and $\mathbf{x}$ is a unit vector, we obtain

$$\begin{aligned} & \left\langle \sum_{i=1}^n \langle \mathbf{x}, \mathbf{v}_i \rangle \lambda_i \mathbf{v}_i, \sum_{i=1}^n \langle \mathbf{x}, \mathbf{v}_i \rangle \mathbf{v}_i \right\rangle \\ &= \sum_{i=1}^n |\langle \mathbf{x}, \mathbf{v}_i \rangle|^2 \lambda_i \\ &\leq \sum_{i=1}^n |\langle \mathbf{x}, \mathbf{v}_i \rangle|^2 \lambda_n \\ &= \lambda_n. \end{aligned}$$

The equality holds if and only if $\lambda_i = \lambda_n$ for all i such that $\langle \mathbf{v}_i, \mathbf{x} \rangle \neq 0$. $\square$

Remark

One can similarly prove that λ_1 is the minimum possible Rayleigh quotient, and that

$$\lambda_i = \min_{\mathbf{x} \perp \psi_1, \dots, \psi_{i-1}} \frac{\langle \mathbf{M}\mathbf{x}, \mathbf{x} \rangle}{\langle \mathbf{x}, \mathbf{x} \rangle},$$

and that one can choose the eigenvectors by

$$\psi_i = \arg \min_{\mathbf{x} \perp \psi_1, \dots, \psi_{i-1}} \frac{\langle \mathbf{M}\mathbf{x}, \mathbf{x} \rangle}{\langle \mathbf{x}, \mathbf{x} \rangle}.$$

Remark

One can similarly prove that λ_1 is the minimum possible Rayleigh quotient, and that

$$\lambda_i = \min_{x \perp \psi_1, \dots, \psi_{i-1}} \frac{x^T M x}{x^T x},$$

and that one can choose the eigenvectors by

$$\psi_i = \arg \min_{x \perp \psi_1, \dots, \psi_{i-1}} \frac{x^T M x}{x^T x}.$$

Moral

The range of the Rayleigh quotient is exactly the range of the eigenvalues of the matrix M .

Example

Consider the matrix

$$\mathbb{A} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}$$

for which the exact eigenvalues are $\lambda_1 = 1, \lambda_2 = 2, \lambda_3 = 3$, with

corresponding eigenvectors $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \mathbf{v}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$

WLOG, consider any unit vector $\mathbf{x} = \begin{bmatrix} a \\ b \\ c \end{bmatrix}, a^2 + b^2 + c^2 = 1$. Evaluate the Rayleigh quotient of $\mathbb{A}$ with respect to $\mathbf{x}$:

$$\frac{\langle \mathbb{A}\mathbf{x}, \mathbf{x} \rangle}{\langle \mathbf{x}, \mathbf{x} \rangle} = \langle \mathbb{A}\mathbf{x}, \mathbf{x} \rangle = a^2 + 2b^2 + 3c^2 \leq 3$$

where the equality holds if and only if $\mathbf{x} = \mathbf{v}_3$.

On the other hand,

$$\frac{\langle \mathbb{A}\mathbf{x}, \mathbf{x} \rangle}{\langle \mathbf{x}, \mathbf{x} \rangle} = \langle \mathbb{A}\mathbf{x}, \mathbf{x} \rangle = a^2 + 2b^2 + 3c^2 \geq 1$$

where the equality holds if and only if $\mathbf{x} = \mathbf{v}_1$.

In addition, when $\mathbf{x} = \mathbf{v}_2$, the value of the Rayleigh quotient equals to the corresponding eigenvalue λ_2 :

$$\frac{\langle \mathbb{A}\mathbf{x}, \mathbf{x} \rangle}{\langle \mathbf{x}, \mathbf{x} \rangle} = \langle \mathbb{A}\mathbf{x}, \mathbf{x} \rangle = 0^2 + 2(1^2) + 3(0^2) = 2 = \lambda_2$$

Rayleigh quotient for complex values

Definition (Rayleigh Quotion)

For **complex** matrices and vectors, the condition of being symmetric extends to being **Hermitian**, and the usual transpose x^T to the **conjugate transpose** x^* :

$$R(\mathbb{M}, \mathbf{x}) = \frac{\mathbf{x}^* \mathbb{M} \mathbf{x}}{\mathbf{x}^* \mathbf{x}} = \frac{\langle \mathbb{M} \mathbf{x}, \mathbf{x} \rangle}{\langle \mathbf{x}, \mathbf{x} \rangle}$$

Definition (Hermitian Matrix)

A Hermitian matrix (or self-adjoint matrix) is a complex square matrix that is equal to its own conjugate transpose—that is, the element in the i -th row and j -th column is equal to the complex conjugate of the element in the j -th row and i -th column.

Rayleigh quotient for complex values

Example

Consider the following matrix:

$$\begin{bmatrix} 0 & a - ib & c - id \\ a + ib & 1 & m - in \\ c + id & m + in & 2 \end{bmatrix}$$

The diagonal elements must be real, as they must be their own complex conjugate.

3.2 Numerical schemes for finding eigenvalues and eigenvectors

- In most introductory linear algebra classes, one computes eigenvalues as roots of a characteristic polynomial.
- For most problems, this is a bad idea: the roots of the characteristic polynomial are often very sensitive to changes in the polynomial coefficients even when they correspond to well-conditioned eigenvalues.

Notations

Given a symmetric matrix $\mathbb{A} \in \mathbb{R}^{n \times n}$, number the eigenvalues so that

$$|\lambda_1| \geq |\lambda_2| \leq \cdots \geq |\lambda_n|.$$

Denote the corresponding eigenvectors as

$$\{\psi_1, \psi_2, \dots, \psi_n\}.$$

By the definition, each eigenvalue and its corresponding eigenvector satisfy

$$\mathbb{A}\psi_i = \lambda_i\psi_i.$$

1. Power iteration

Power iteration

Main Idea

Given a symmetric matrix $\mathbb{A} \in \mathbb{R}^{n \times n}$, the power iteration is expected to produce a sequence $\mathbf{v}^{(i)}$ that converges to ψ_1 .

Algorithm 1 Power iteration

```
1:  $\mathbf{v}^{(0)}$  = some vector with  $\|\mathbf{v}^{(0)}\| = 1$ 
2: for  $k = 1, 2, \dots$  do
3:    $\mathbf{w} = \mathbb{A}\mathbf{v}^{(k-1)}$                                      % apply  $\mathbb{A}$ 
4:    $\mathbf{v}^{(k)} = \mathbf{w} / \|\mathbf{w}\|$                              % normalize
5:    $\lambda^{(k)} = \left(\mathbf{v}^{(k)}\right)^T \mathbb{A}\mathbf{v}^{(k)}$  % Rayleigh quotient
6: end for
```

What is the computational complexity of Algorithm 1?

Analysis of power iteration

Write $\mathbf{v}^{(0)}$ as a linear combination of the orthonormal eigenvectors ψ_i :

$$\mathbf{v}^{(0)} = a_1\psi_1 + a_2\psi_2 + \cdots + a_n\psi_n.$$

Since $\mathbf{v}^{(k)}$ is a multiple of $\mathbb{A}^k \mathbf{v}^{(0)}$, we have for some constants c_k

$$\begin{aligned}\mathbf{v}^{(k)} &= c_k \mathbb{A}^k \mathbf{v}^{(0)} \\ &= c_k \left(a_1 \lambda_1^k \psi_1 + a_2 \lambda_2^k \psi_2 + \cdots + a_n \lambda_n^k \psi_n \right) \\ &= c_k \lambda_1^k \left(a_1 \psi_1 + a_2 (\lambda_2/\lambda_1)^k \psi_2 + \cdots + a_n (\lambda_n/\lambda_1)^k \psi_n \right).\end{aligned}$$

From here we obtain the convergence conclusion.

Convergence of power iteration

Theorem

Suppose $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$ and $\psi_1^T \mathbf{v}^{(0)} \neq 0$. Then the iterates of the algorithm satisfy

$$\left\| \mathbf{v}^{(k)} - (\pm \psi_1) \right\| = O \left(\left| \frac{\lambda_2}{\lambda_1} \right|^k \right), \quad \left| \lambda^{(k)} - \lambda_1 \right| = O \left(\left| \frac{\lambda_2}{\lambda_1} \right|^{2k} \right)$$

as $k \rightarrow \infty$.

Power iteration is of limited use for several reasons:

- It can find only the eigenvector corresponding to the largest eigenvalue.
- the convergence is **linear**, reducing the error only by a constant factor $\approx \left| \frac{\lambda_2}{\lambda_1} \right|$ at each iteration.
- **The quality of this factor depends on having the largest eigenvalue that is significantly larger than the others.**

Example

Consider the matrix

$$\mathbb{A} = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 3 & 1 \\ 1 & 1 & 4 \end{bmatrix}$$

for which the exact eigenvalues are $\lambda_1 = 5.214319743377$, $\lambda_2 \approx 2.4608$ and $\lambda_3 \approx 1.3249$, with corresponding eigenvectors

$$\mathbf{v}_1 \approx \begin{bmatrix} -0.3971 \\ -0.5207 \\ -0.7558 \end{bmatrix}, \mathbf{v}_2 \approx \begin{bmatrix} 0.2332 \\ 0.7392 \\ -0.6318 \end{bmatrix} \text{ and } \mathbf{v}_3 \approx \begin{bmatrix} 0.8877 \\ -0.4271 \\ -0.1721 \end{bmatrix}$$

The largest eigenvalue is $\lambda_1 \approx 5.2143$ and corresponds to any eigenvector

proportional to $\mathbf{v}_1 \approx \begin{bmatrix} -0.3971 \\ -0.5207 \\ -0.7558 \end{bmatrix}$.

Proof.

We begin with an initial guess, $\mathbf{v}^{(0)} := \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$.

Then, the first iteration yields $\mathbf{v}^{(1)} \approx \begin{bmatrix} 0.4558 \\ 0.5698 \\ 0.6838 \end{bmatrix}$, $\lambda^{(1)} \approx 5.1818$.

The second iteration, $\mathbf{v}^{(2)} \approx \begin{bmatrix} 0.4171 \\ 0.5488 \\ 0.7244 \end{bmatrix}$, $\lambda^{(2)} \approx 5.208192771084338 \dots$.

The 10th iteration, $\mathbf{v}^{(10)} \approx \begin{bmatrix} 0.3971 \\ 0.5207 \\ 0.7557 \end{bmatrix}$, $\lambda^{(10)} \approx 5.214319709113863 \dots$.

After 15th iterations, power iteration has produced a result accurate to ten digits. □

Implementation of power iteration

```
%% Sample Matlab code  
%Power iteration  
A=[2 1 1 ;1 3 1;1 1 4];  
v0=[1;1;1]/sqrt(3);  
for k=1:15  
    w=A*v0;  
    v0=w/sqrt(w'*w);  
    lambda0=v0'*A*v0;  
end
```

2. Inverse iteration

Inverse iteration

Fact

For any $\mu \in \mathbb{R}$ that is not an eigenvalue of $\mathbb{A}$, the eigenvectors of $(\mathbb{A} - \mu\mathbb{I})^{-1}$ are the same as the eigenvectors of $\mathbb{A}$, and the corresponding eigenvalues are $\{(\lambda_j - \mu)^{-1}\}$, where $\{\lambda_j\}$ are the eigenvalues of A . This suggests the idea for inverse iteration.

Algorithm 2 Inverse iteration

- 1: $\mathbf{v}^{(0)}$ = some vector with $\|\mathbf{v}^{(0)}\| = 1$
 - 2: **for** $k = 1, 2, \dots$ **do**
 - 3: Solve $(\mathbb{A} - \mu\mathbb{I})\mathbf{w} = \mathbf{v}^{(k-1)}$ for $\mathbf{w}$ % apply $(\mathbb{A} - \mu\mathbb{I})^{-1}$
 - 4: $\mathbf{v}^{(k)} = \mathbf{w} / \|\mathbf{w}\|$ % normalize
 - 5: $\lambda^{(k)} = \left(\mathbf{v}^{(k)}\right)^T \mathbb{A} \mathbf{v}^{(k)}$ % Rayleigh quotient
 - 6: **end for**
-

What is the computational complexity of Algorithm 2?

Analysis of inverse iteration

Theorem

Suppose λ_J is the closest eigenvalue to μ and λ_K is the second closest, that is, $|\mu - \lambda_J| < |\mu - \lambda_K| \leq |\mu - \lambda_j|$ for each $j \neq J$ or K . Furthermore, suppose $\psi'_J \mathbf{v}^{(0)} \neq 0$. Then the iterates of Algorithm 2 satisfy

$$\|\mathbf{v}^{(k)} - (\pm \psi_J)\| = O\left(\left|\frac{\mu - \lambda_J}{\mu - \lambda_K}\right|^k\right), \quad |\lambda^{(k)} - \lambda_J| = O\left(\left|\frac{\mu - \lambda_J}{\mu - \lambda_K}\right|^{2k}\right)$$

as $k \rightarrow \infty$.

- Like power iteration, inverse iteration exhibits only **linear** convergence.
- Fortunately, the convergence rate can be controlled: if μ is much closer to one eigenvalue of $\mathbb{A}$ than to the others, then the largest eigenvalue of $(\mathbb{A} - \mu\mathbb{I})^{-1}$ will be much larger than the rest.
- **Inverse iteration is a standard tool to calculate one or more eigenvectors of a matrix if the eigenvalues are known.**

Example

Consider the matrix

$$\mathbb{A} = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 3 & 1 \\ 1 & 1 & 4 \end{bmatrix}$$

for which the exact eigenvalues are $\lambda_1 = 5.214319743377$, $\lambda_2 \approx 2.4608$ and $\lambda_3 \approx 1.3249$, with corresponding eigenvectors

$$\mathbf{v}_1 \approx \begin{bmatrix} -0.3971 \\ -0.5207 \\ -0.7558 \end{bmatrix}, \mathbf{v}_2 \approx \begin{bmatrix} 0.2332 \\ 0.7392 \\ -0.6318 \end{bmatrix} \text{ and } \mathbf{v}_3 \approx \begin{bmatrix} 0.8877 \\ -0.4271 \\ -0.1721 \end{bmatrix}$$

The largest eigenvalue is $\lambda_1 \approx 5.2143$ and corresponds to any eigenvector

proportional to $\mathbf{v}_1 \approx \begin{bmatrix} -0.3971 \\ -0.5207 \\ -0.7558 \end{bmatrix}$.

Proof.

We begin with an initial guess, $\mathbf{v}^{(0)} := \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$. and $\mu = 5$.

Then, the first iteration yields

$$\mathbf{v}^{(1)} \approx \begin{bmatrix} 0.3841 \\ 0.5121 \\ 0.7682 \end{bmatrix}, \lambda^{(1)} \approx 5.213114754098362.$$

The second iteration, $\mathbf{v}^{(2)} \approx \begin{bmatrix} 0.3980 \\ 0.5215 \\ 0.7548 \end{bmatrix}, \lambda^{(2)} \approx 5.214312617702449 \dots$

The 5th iteration, $\mathbf{v}^{(10)} \approx \begin{bmatrix} 0.3971 \\ 0.5201 \\ 0.7558 \end{bmatrix}, \lambda^{(5)} \approx 5.214319743375390 \dots$

After 6th iterations, power iteration has produced a result accurate to 12 digits. □

Exercise: Implementation of Inverse iteration

You can run the following matlab code using Inverse iteration, and compare the results.

```
%% Sample Matlab code  
%Inverse iteration  
A=[2 1 1 ;1 3 1;1 1 4];  
v0=[1;1;1]/sqrt(3);mu0=5;  
for k=1:6  
    w=(A-mu0*eye(3))\v0;  
    v0=w/sqrt(w'*w);  
    lambda0=v0'*A*v0;  
end
```

3. Rayleigh quotient iteration

Rayleigh quotient iteration

Main Idea

Rayleigh quotient iteration is an eigenvalue algorithm which extends the idea of the inverse iteration by using the Rayleigh quotient to obtain increasingly accurate eigenvalue estimates.

Algorithm 3 Rayleigh quotient iteration

- 1: $\mathbf{v}^{(0)}$ = some vector with $\|\mathbf{v}^{(0)}\| = 1$
 - 2: $\lambda^{(0)} = \left(\mathbf{v}^{(0)}\right)^T \mathbb{A} \mathbf{v}^{(0)}$ = corresponding Rayleigh quotient
 - 3: **for** $k = 1, 2, \dots$ **do**
 - 4: Solve $\left(\mathbb{A} - \lambda^{(k-1)}\mathbb{I}\right) \mathbf{w} = \mathbf{v}^{(k-1)}$ for $\mathbf{w}$ % apply $(\mathbb{A} - \lambda^{(k-1)}\mathbb{I})^{-1}$
 - 5: $\mathbf{v}^{(k)} = \mathbf{w} / \|\mathbf{w}\|$ % normalize
 - 6: $\lambda^{(k)} = \left(\mathbf{v}^{(k)}\right)^T \mathbb{A} \mathbf{v}^{(k)}$ % Rayleigh quotient
 - 7: **end for**
-

What is the computational complexity of Algorithm 3?

Convergence of Rayleigh quotient iteration

The convergence of this algorithm is spectacular: each iteration triples the number of digits of accuracy.

Theorem

Rayleigh quotient iteration converges to an eigenvalue/eigenvector pair for all except a set of measure zero of starting vectors $\mathbf{v}^{(0)}$. When it converges, the convergence is ultimately cubic in the sense that if λ_J is an eigenvalue of $\mathbb{A}$ and $\mathbf{v}^{(0)}$ is sufficiently close to the eigenvector ψ_J , then

$$\left\| \mathbf{v}^{(k+1)} - (\pm \psi_J) \right\| = \mathcal{O} \left(\left\| \mathbf{v}^{(k)} - (\pm \psi_J) \right\|^3 \right)$$

and

$$\left| \lambda^{(k+1)} - \lambda_J \right| = \mathcal{O} \left(\left| \lambda^{(k)} - \lambda_J \right|^3 \right)$$

as $k \rightarrow \infty$.

1/4.

Here is a proof that if convergence occurs, it is ultimately cubic. For simplicity, we assume that the eigenvalue λ_J is simple. By the algorithm, if

$$\|\mathbf{v}^{(k)} - \boldsymbol{\psi}_J\| \leq \epsilon$$

for sufficiently small ϵ , then $\mathbf{v}^{(k)}$ can be represented by

$$\mathbf{v}^{(k)} = \alpha \boldsymbol{\psi}_J + \sqrt{1 - \alpha^2} \boldsymbol{\psi}_J^\perp \text{ with } \alpha \in [1 - \epsilon^2/2, 1].$$

Here, $\boldsymbol{\psi}_J^\perp$ denote a vector such that $(\boldsymbol{\psi}_J^\perp, \boldsymbol{\psi}_J) = 0$.

Note that the Rayleigh quotient yields an eigenvalue estimate $\lambda^{(k)}$ with

$$\begin{aligned} |\lambda^{(k)} - \lambda_J| &= \left| \lambda_J \left(\boldsymbol{\psi}_J - \mathbf{v}^{(k)}, \boldsymbol{\psi}_J - \mathbf{v}^{(k)} \right) - \left(\mathbb{A}(\boldsymbol{\psi}_J - \mathbf{v}^{(k)}), \boldsymbol{\psi}_J - \mathbf{v}^{(k)} \right) \right| \\ &= O(\epsilon^2). \end{aligned}$$

2/4.

Let k be sufficiently large so that $\lambda^{(k)}$ is very close to λ_J , then by the property of the matrix norm associated with the vector norm, we obtain

$$\|(\mathbb{A} - \lambda^{(k)}\mathbb{I})^{-1}(\mathbf{v}^{(k)} - \psi_J)\| \leq 2\delta^{-1}\|\mathbf{v}^{(k)} - \psi_J\|. \quad (1)$$

Here, $\delta = \min_{j \neq J} |\lambda_J - \lambda_j|$ denotes the distance between λ_J and the remaining eigenvalues.

Note that

$$\begin{aligned} \mathbf{w} &= (\mathbb{A} - \lambda^{(k)}\mathbb{I})^{-1}\mathbf{v}^{(k)} \\ \|\mathbf{w}\| &\approx (\lambda_J - \lambda^{(k)})^{-1} \\ (\mathbb{A} - \lambda^{(k)}\mathbb{I})(\mathbf{v}^{(k+1)} - \psi_J) &\approx (\lambda_J - \lambda^{(k)})(\mathbf{v}^{(k)} - \psi_J). \end{aligned}$$



3/4.

Plug into (1), we obtain

$$\|\mathbf{v}^{(k+1)} - \psi_J\| \leq 2\delta^{-1}(\lambda_J - \lambda^{(k)})\|\mathbf{v}^{(k)} - \psi_J\|$$

If we now take one step of inverse iteration to obtain a new $\mathbf{v}^{(k+1)}$ from $\mathbf{v}^{(k)}$ and $\lambda^{(k)}$, then

$$\|\mathbf{v}^{(k+1)} - \psi_J\| = \mathcal{O}\left(|\lambda^{(k)} - \lambda_J|\|\mathbf{v}^{(k)} - \psi_J\|\right) = \mathcal{O}(\epsilon^3)$$



4/4.

Moreover, the constants implicit in the $\mathcal{O}$ symbols are uniform throughout sufficiently small neighborhoods of λ_J and ψ_J . Thus we have convergence in the following pattern:

$$\begin{array}{ccc}
 \left\| \mathbf{v}^{(k)} - (\pm \psi_J) \right\| & & \left| \lambda^{(k)} - \lambda_J \right| \\
 \epsilon & \rightarrow \mathcal{O}(\epsilon^2) & \\
 \downarrow \swarrow & & \\
 \mathcal{O}(\epsilon^3) & \rightarrow \mathcal{O}(\epsilon^6) & \\
 \downarrow \swarrow & & \\
 \mathcal{O}(\epsilon^9) & \rightarrow & \mathcal{O}(\epsilon^{18}) \\
 \vdots & \vdots &
 \end{array}$$

The convergence results follow from this uniformity. □

Example

Consider the matrix

$$\mathbb{A} = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 3 & 1 \\ 1 & 1 & 4 \end{bmatrix}$$

for which the exact eigenvalues are $\lambda_1 = 5.214319743377$, $\lambda_2 \approx 2.4608$ and $\lambda_3 \approx 1.3249$, with corresponding eigenvectors

$$\mathbf{v}_1 \approx \begin{bmatrix} -0.3971 \\ -0.5207 \\ -0.7558 \end{bmatrix}, \mathbf{v}_2 \approx \begin{bmatrix} 0.2332 \\ 0.7392 \\ -0.6318 \end{bmatrix} \text{ and } \mathbf{v}_3 \approx \begin{bmatrix} 0.8877 \\ -0.4271 \\ -0.1721 \end{bmatrix}$$

The largest eigenvalue is $\lambda_1 \approx 5.2143$ and corresponds to any eigenvector

proportional to $\mathbf{v}_1 \approx \begin{bmatrix} -0.3971 \\ -0.5207 \\ -0.7558 \end{bmatrix}$.

Proof.

We begin with an initial guess $\mathbf{v}^{(0)} := \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$, $\lambda^{(0)} = 5$

Then, the first iteration yields $\mathbf{v}^{(1)} \approx \begin{bmatrix} 0.3841 \\ 0.5121 \\ 0.7682 \end{bmatrix}$, $\lambda^{(1)} \approx 5.2131$.

The second iteration, $\mathbf{v}^{(2)} \approx \begin{bmatrix} 0.3971 \\ 0.5207 \\ 0.7558 \end{bmatrix}$, $\lambda^{(2)} \approx 5.214319743184 \dots$

After only two iterations, Rayleigh quotient iteration has produced a result accurate to ten digits. Three more iterations would increase this figure to about 270 digits, if our machine precision were high enough. □

Implementation of Rayleigh quotient iteration

```
%% Sample Matlab code  
%rayleigh-quotient iteration  
A=[2 1 1 ;1 3 1;1 1 4];  
v0=[1;1;1]/sqrt(3);  
lambda0=v0'*A*v0;  
for k=1:3  
    w=(A-lambda0*eye(3))\v0;  
    v0=w/sqrt(w'*w);  
    lambda0=v0'*A*v0;  
end
```


We close this section with a note on the running cost of the three iterations we have described. Suppose $\mathbb{A} \in \mathbb{R}^{m \times m}$ is a full matrix.

- Each step of power iteration involves a matrix-vector multiplication, requiring $\mathcal{O}(m^2)$ flops.
- Each step of inverse iteration involves the solution of a linear system, which might seem to require $\mathcal{O}(m^3)$ flops, but this figure reduces to $\mathcal{O}(m^2)$ if the matrix is processed in advance by LU or QR factorization or another method.
- In the case of Rayleigh quotient iteration, the matrix to be inverted changes at each step, and beating $\mathcal{O}(m^3)$ flops per step is not so straightforward.

3.3 Formulation using Lagrange multipliers

Moral

As indicated in the previous lecture, the Rayleigh quotient is maximized by the eigenvector with the largest eigenvalue. This result can be arrived at by the method of Lagrange multipliers.

Recall that the Rayleigh quotient is constant under scaling $\mathbf{x} \rightarrow c\mathbf{x}$, where $c \in \mathbb{R}$ is a scalar

$$R(\mathbb{M}, c\mathbf{x}) = \frac{\langle M c\mathbf{x}, c\mathbf{x} \rangle}{\langle c\mathbf{x}, c\mathbf{x} \rangle} = \frac{c^2}{c^2} \frac{\langle \mathbb{M}\mathbf{x}, \mathbf{x} \rangle}{\langle \mathbf{x}, \mathbf{x} \rangle} = R(\mathbb{M}, \mathbf{x}).$$

Because of this invariance, it is sufficient to study the special case $\|\mathbf{x}\|^2 = \mathbf{x}^T \mathbf{x} = 1$.

Method of Lagrange multipliers

Moral

The method of Lagrange multipliers is a strategy for finding the local maxima and minima of a function subject to equality constraints. The relationship between the gradient of the function and gradients of the constraints leads to the Lagrangian function.

Method of Lagrange multipliers

To find the maximum or minimum of a function $f(x)$ subjected to the equality constraint $g(x) = 0$, form the Lagrangian function

$$\mathcal{L}(x, \lambda) = f(x) + \lambda g(x)$$

and find the stationary points of $\mathcal{L}$ considered as a function of x and the Lagrange multiplier λ ; this means that all partial derivatives should be zero.

Maximize the Rayleigh quotient

Problem

Our problem is to find the critical points of the function

$$R(\mathbb{M}, \mathbf{x}) = \mathbf{x}^T \mathbb{M} \mathbf{x}$$

subject to the constraint

$$\|\mathbf{x}\|^2 = \mathbf{x}^T \mathbf{x} = 1.$$

In other words, it is to find the critical points of

$$\mathcal{L}(\mathbf{x}, \lambda) = \mathbf{x}^T \mathbb{M} \mathbf{x} - \lambda (\mathbf{x}^T \mathbf{x} - 1)$$

where λ is a Lagrange multiplier.

Apply the method of Lagrange multipliers

Solution

The stationary points of $\mathcal{L}(\mathbf{x}, \lambda)$ occur at

$$\begin{aligned}\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \lambda) &= 0 \\ \Rightarrow 2\mathbf{x}^T \mathbb{M} - 2\lambda \mathbf{x}^T &= 0 \\ \Rightarrow 2\mathbb{M}\mathbf{x} - 2\lambda \mathbf{x} &= 0 \\ \Rightarrow \mathbb{M}\mathbf{x} &= \lambda \mathbf{x}\end{aligned}$$

At the stationary points,

$$R(\mathbb{M}, \mathbf{x}) = \frac{\langle \mathbb{M}\mathbf{x}, \mathbf{x} \rangle}{\langle \mathbf{x}, \mathbf{x} \rangle} = \frac{\langle \lambda \mathbf{x}, \mathbf{x} \rangle}{\langle \mathbf{x}, \mathbf{x} \rangle} = \lambda$$

Summary

- The eigenvectors $\mathbf{x}_1, \dots, \mathbf{x}_n$ of $\mathbb{M}$ are the critical points of the Rayleigh quotient and their corresponding eigenvalues $\lambda_1, \dots, \lambda_n$ are the stationary values of $\mathcal{L}$.
- This property is the basis for principal components analysis (PCA) and canonical correlation.

3.4 Principal Components Analysis (PCA)

1. Motivation and main idea

Dimensionality Reduction

Dimensionality reduction is the process of taking data in a high dimensional space and mapping it into a new space whose dimensionality is much smaller. The motivation for it includes,

- high dimensional data impose computational challenges¹
- high dimensional data can lead to poor generalization abilities of the learning algorithm
- Dimensionality reduction can be used for interpretability of the data, for finding meaningful structure of the data, and for illustration purposes.

We will focus on Principal Component Analysis (PCA) in this section, which is one of the methods to realize dimensionality reduction.

¹e.g., matrix-vector multiplication, 'curse of dimensionality'

Main ingredients of Principal Component Analysis (PCA)

Let $\{\mathbf{x}_1, \dots, \mathbf{x}_n\} \subset \mathbb{R}^d$ be the original data with dimension $d \gg 1$ huge. Then the main idea of dimensionality reduction is first performing the **compression** by applying a linear transformation $\mathbb{W} \in \mathbb{R}^{k \times d}$ with $k \ll d$, which induces the mapping $\mathbf{x}_i \mapsto \mathbb{W}\mathbf{x}_i$. Then perform a **recovery** by applying another linear transformation $\mathbb{U} \in \mathbb{R}^{d \times k}$, which induces a good approximation to each data $\mathbf{x}_i \approx \mathbb{U}(\mathbb{W}\mathbf{x}_i)$. The aim of PCA is to make the total squared distance between the original and the recovered vectors minimal, i.e., we aim to solve the following optimization problem²,

$$\arg \min_{\substack{\mathbb{W} \in \mathbb{R}^{k \times d} \\ \mathbb{U} \in \mathbb{R}^{d \times k}}} \sum_{i=1}^n \|\mathbf{x}_i - \mathbb{U}\mathbb{W}\mathbf{x}_i\|_2^2.$$

²One can prove that the columns of $\mathbb{U}$ are orthonormal, and $\mathbb{W} = \mathbb{U}^T$.

2. Interpretation of PCA as Variance Maximization

Mathematical intuition

Goal

We want to summarize a collection of vectors in d -dimensional space by projecting down into a lower dimensional subspace, while the information in the original data is “essentially” preserved even after the reduction.

Moral

The simplest way to find the principal components is by finding the projections which maximize the variance.

Remark

It might sound more plausible to look for the projection with the smallest mean-squared distance between the original vectors and their projections on to the principal components. In fact, this turns out to be equivalent to maximizing the variance.

Elementaries on probability

Definition (Multivariate random variable/random vector)

A multivariate random variable is a column vector $\mathbf{x} = (x_1, \dots, x_n)^\top$ (or its transpose, which is a row vector) whose components are scalar-valued random variables on the same probability space as each other.

Definition (Mean)

The expected value or mean of a random vector $\mathbf{x}$ is a fixed vector $\mathbb{E}[\mathbf{x}]$ whose elements are the expected values of the respective random variables

$$\mathbb{E}[\mathbf{x}] := (\mathbb{E}[x_1], \dots, \mathbb{E}[x_n])^\top.$$

We say a family of vectors have **zero mean** if their mean is the zero vector.

Comment

It is sufficient to consider vectors with **zero mean**, otherwise we can adjust each vector by subtracting the mean vector.

Multivariate random variable

Definition (Covariance)

The covariance matrix (also called second central moment or variance-covariance matrix) of an $n \times 1$ random vector is an $n \times n$ matrix whose $(i, j)^{\text{th}}$ element is the covariance between the i^{th} and the j^{th} random variables.

$$\begin{aligned}\text{Var}[\mathbf{x}] &:= \mathbb{E} \left[(\mathbf{x} - \mathbb{E}[\mathbf{x}])(\mathbf{x} - \mathbb{E}[\mathbf{x}])^T \right] \\ &= \mathbb{E} \left[\mathbf{x}\mathbf{x}^T \right] - \mathbb{E}[\mathbf{x}]\mathbb{E}[\mathbf{x}]^T.\end{aligned}$$

Exercise

Can you prove that the covariance matrix of any random vector is a symmetric and positive semidefinite matrix?

Seeking the principal components

Let $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n \in \mathbb{R}^d$ be d -dimensional vectors, and let $\mathbf{x}$ be a random vector over $(\Omega, 2^\Omega, \mathbb{P})$ distributed according to the uniform distribution over $\Omega := \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$, i.e.,

$$\mathbb{P}(\mathbf{x} = \mathbf{x}_i) = \frac{1}{n} \text{ for all } i = 1, \dots, n.$$

Without loss of generality, we assume $\mathbf{x}$ has vanishing mean, i.e.,

$$\mathbb{E}[\mathbf{x}] = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i = \mathbf{0}.$$

Apply the definition of the covariance for a random vector, we derive,

$$\text{Var}[\mathbf{x}] = \mathbb{E}[\mathbf{x}\mathbf{x}^T] = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^T.$$

One-dimensional projection: objective

We want to find a **unit vector** $\mathbf{w} \in \mathbb{R}^d$ such that the mean of the squares of the discrepancy between the random variable $\mathbf{x}$ and its projection onto the line determined by $\mathbf{w}$ that passes the origin is minimized. We will show this is **equivalent** to **maximize the variance of the random variable** $\langle \mathbf{w}, \mathbf{x} \rangle$.

Note that the random variable $\langle \mathbf{w}, \mathbf{x} \rangle$ also has a vanishing mean, since

$$\mathbb{E}[\langle \mathbf{w}, \mathbf{x} \rangle] = \langle \mathbf{w}, \mathbb{E}[\mathbf{x}] \rangle = 0.$$

Consequently, its variance of $\langle \mathbf{w}, \mathbf{x} \rangle$ equals to its second moment, i.e.,

$$\text{Var}(\langle \mathbf{w}, \mathbf{x} \rangle) = \mathbb{E}[|\langle \mathbf{w}, \mathbf{x} \rangle|^2].$$

Minimizing projection residuals

- Specify the line by a unit vector along it, say, $\mathbf{w}$.
- The projection of a data vector $\mathbf{x}_i$ on to the line is $\langle \mathbf{w}, \mathbf{x}_i \rangle^3$, which is a scalar. This is the distance of the projection from the origin.
- The actual coordinate in d-dimensional space is $\mathcal{P}_1(\mathbf{x}_i) := \langle \mathbf{w}, \mathbf{x}_i \rangle \mathbf{w}$.

³See Claim 15.1, Section 15.1 of the reference book by Shalev-Shwartz and Ben-David for the proof

How big is the error or residual?

Residual of the projection

If we try to use our projected vectors instead of our original vectors, there will be some error. For any vector, say $\mathbf{x}_i$, the residual is

$$\begin{aligned}\|\mathbf{x}_i - \mathcal{P}_1(\mathbf{x}_i)\|^2 &= \langle \mathbf{x}_i - \langle \mathbf{x}_i, \mathbf{w} \rangle \mathbf{w}, \mathbf{x}_i - \langle \mathbf{x}_i, \mathbf{w} \rangle \mathbf{w} \rangle \\ &= \langle \mathbf{x}_i, \mathbf{x}_i \rangle - \langle \mathbf{x}_i, \langle \mathbf{x}_i, \mathbf{w} \rangle \mathbf{w} \rangle \\ &\quad - \langle \mathbf{w}, \mathbf{x}_i \rangle \langle \mathbf{x}_i, \mathbf{w} \rangle + \langle \langle \mathbf{w}, \mathbf{x}_i \rangle \mathbf{w}, \langle \mathbf{w}, \mathbf{x}_i \rangle \mathbf{w} \rangle \\ &= \|\mathbf{x}_i\|^2 - 2|\langle \mathbf{w}, \mathbf{x}_i \rangle|^2 + |\langle \mathbf{w}, \mathbf{x}_i \rangle|^2 \langle \mathbf{w}, \mathbf{w} \rangle \\ &= \|\mathbf{x}_i\|^2 - |\langle \mathbf{w}, \mathbf{x}_i \rangle|^2.\end{aligned}$$

Add the residuals up across all the vectors

Definition (Mean-squared error)

The mean-squared error (MSE) of a predictor (i.e., a function mapping arbitrary inputs to a sample of values of some random variable) is the mean of the squares of the errors.

Following the definition of MSE, we have

$$\begin{aligned}\text{MSE}(\mathbf{w}) &= \frac{1}{n} \sum_{i=1}^n \left(\|\mathbf{x}_i\|^2 - |\langle \mathbf{w}, \mathbf{x}_i \rangle|^2 \right) \\ &= \frac{1}{n} \left(\sum_{i=1}^n \|\mathbf{x}_i\|^2 - \sum_{i=1}^n |\langle \mathbf{w}, \mathbf{x}_i \rangle|^2 \right)\end{aligned}$$

Observation 1

The first summation doesn't depend on $\mathbf{w}$. To make the MSE small, what we must do is make the second sum big.

The first principal component

Goal

We want to solve

$$\mathbf{w}^* =: \arg \max_{\substack{\mathbf{w} \in \mathbb{R}^d \\ \|\mathbf{w}\|=1}} \mathbb{E} [|\langle \mathbf{w}, \mathbf{x} \rangle|^2] .$$

$\mathbf{w}^*$ is the so-called first principal component.

Observation 2

Since we've just seen that the mean of the projections is zero, minimizing the residual sum of squares turns out to be equivalent to maximizing the variance of the projections.

Multiple principal components

In general we don't want to project on to just one vector, but on to multiple principal components. If those components are **orthogonal unit vectors** $\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_s$, then the image of $\mathbf{x}_i$ is its projection into the space spanned by these vectors,

$$\begin{aligned}\mathcal{P}_s(\mathbf{x}_i) &:= \sum_{j=1}^s \langle \mathbf{x}_i, \mathbf{w}_j \rangle \mathbf{w}_j \\ &= \mathbf{U}\mathbf{W}\mathbf{x}_i\end{aligned}$$

with

$$\mathbf{U} = [\mathbf{w}_1, \dots, \mathbf{w}_s] \text{ and } \mathbf{W} = \mathbf{U}^T.$$

Exercise

1 Calculate

$$\|\mathbf{x}_i - \mathcal{P}_s(\mathbf{x}_i)\|_2^2.$$

2 Calculate

$$\sum_{i=1}^n \|\mathbf{x}_i - \mathcal{P}_s(\mathbf{x}_i)\|_2^2.$$

Connection to our main question

We can prove that the following two problems are equivalent,

1 Solve

$$[\mathbf{w}_1^*, \dots, \mathbf{w}_s^*] := \arg \min_{\substack{\mathbf{w}_1, \dots, \mathbf{w}_s \in \mathbb{R}^d \\ \|\mathbf{w}_1\|, \dots, \|\mathbf{w}_s\| = 1}} \sum_{i=1}^n \left\| \mathbf{x}_i - \sum_{j=1}^s \langle \mathbf{x}_i, \mathbf{w}_j \rangle \mathbf{w}_j \right\|_2^2.$$

2 Solve

$$[\mathbf{W}^*, \mathbf{U}^*] = \arg \min_{\substack{\mathbf{W} \in \mathbb{R}^{s \times d} \\ \mathbf{U} \in \mathbb{R}^{d \times s}}} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{U}\mathbf{W}\mathbf{x}_i\|_2^2.$$

In the sense that $\mathbf{U}^* = [\mathbf{w}_1^*, \dots, \mathbf{w}_s^*]$ and $\mathbf{W}^* = \mathbf{U}^T$.

3. Implementation of PCA

Terminology

We stack our n data vectors $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ into $\mathbb{X} := [\mathbf{x}_1, \dots, \mathbf{x}_n]^T \in \mathbb{R}^{n \times d}$, then the projections are given by $\mathbb{X}\mathbf{w}$, which is an $n \times 1$ matrix.

The variance is

$$\begin{aligned}\sigma_{\mathbf{w}}^2 &:= \mathbb{E} [|\langle \mathbf{w}, \mathbf{x} \rangle|^2] = \frac{1}{n} \sum_{i=1}^n |\langle \mathbf{w}, \mathbf{x}_i \rangle|^2 \\ &= \frac{1}{n} \langle \mathbb{X}\mathbf{w}, \mathbb{X}\mathbf{w} \rangle \\ &= \frac{1}{n} \mathbf{w}^T \mathbb{X}^T \mathbb{X} \mathbf{w} \\ &= \mathbf{w}^T \frac{\mathbb{X}^T \mathbb{X}}{n} \mathbf{w} \\ &:= \mathbf{w}^T \mathbb{V}_W \mathbf{w}\end{aligned}$$

Maximizing variance

Constrained maximization

We want to choose a unit vector $\mathbf{w}$ so as to maximize $\sigma_{\mathbf{w}}^2$. To do this, we need to make sure that we only look at unit vectors — we need to constrain the maximization. The constraint is that $\|\mathbf{w}\|^2 = 1$.

Apply the method of Lagrange multiplier

The Lagrangian function is

$$\mathcal{L}(\mathbf{w}, \lambda) = \sigma_{\mathbf{w}}^2 - \lambda (\mathbf{w}^T \mathbf{w} - 1)$$

To find the stationary point of it, we take the partial differentiations

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \lambda} &= \mathbf{w}^T \mathbf{w} - 1 \\ \frac{\partial \mathcal{L}}{\partial \mathbf{w}} &= 2\mathbb{V}\mathbf{w} - 2\lambda\mathbf{w}\end{aligned}$$

Apply the method of Lagrange multiplier

Setting the derivatives to zero at the optimum, we get

$$\begin{cases} \mathbf{w}^T \mathbf{w} = 1 \\ \mathbb{V} \mathbf{w} = \lambda \mathbf{w}. \end{cases}$$

Conclusion

Thus, desired vector $\mathbf{w}$ is an eigenvector of the covariance matrix $\mathbb{V}$, and the maximizing vector will be the one associated with the largest eigenvalue λ .

Covariance matrix $\mathbb{V}$

Properties of $\mathbb{V}$

- We know that $\mathbb{V}$ is a $d \times d$ matrix, so it will have d different eigenvectors.
- Since $\mathbb{V}$ is a covariance matrix, so it is symmetric, and then linear algebra tells us that the eigenvectors must be orthogonal to one another.
- Again because $\mathbb{V}$ is a covariance matrix, it is a positive matrix, in the sense that $\mathbf{x}'\mathbb{V}\mathbf{x} \geq 0$ for any $\mathbf{x}$. This tells us that the eigenvalues of $\mathbb{V}$ must all be non-negative.

Conclusion

The eigenvectors of $\mathbb{V}$ are the principal components of the data.

Pseudocode of PCA

```
%% Sample Matlab code for PCA  
%find the first r principal modes  
X=[1 2 1 4 5 -2;0 1 2 0 -1 -0.5;3 0 0 -1 2 -1];  
[n,d]=size(X);  
V=sparse(d,d); r=1;  
V=V+1/n*X'*X;  
[W,D]=eigs(V,r);
```

Question

What is the computational complexity of PCA?

What if $n \ll d$?

- In some situations, the original dimensionality of the data d is much larger than the number of examples n . Then the computational complexity of calculating the PCA solution is $\mathcal{O}(d^3)$ for calculating the eigenvalues of $\mathbb{V}$ plus $\mathcal{O}(nd^2)$ for constructing the matrix $\mathbb{V}$.
- If $n \ll d$, then we can play a trick of calculating the eigenvalue problem of $\tilde{\mathbb{V}} := \frac{\mathbb{X}\mathbb{X}^T}{n}$ instead. Let $(\lambda_i, \mathbf{u}_i)_{i=1}^n$ be the eigen pairs of $\tilde{\mathbb{V}}$ with eigenvalues ordered nondecreasingly. Then $(\lambda_i, \mathbb{X}^T \mathbf{u}_i)_{i=1}^n$ is the eigen pairs for $\mathbb{V}$. In this way, Then the computational complexity of calculating the PCA solution is $\mathcal{O}(n^3)$ for calculating the eigenvalues of $\tilde{\mathbb{V}}$ plus $\mathcal{O}(dn^2)$ for constructing the matrix $\tilde{\mathbb{V}}$.

Connection to our main question

We can prove that the following two problems are equivalent,

1 Solve

$$[\mathbf{w}_1^*, \dots, \mathbf{w}_s^*] := \arg \min_{\substack{\mathbf{w}_1, \dots, \mathbf{w}_s \in \mathbb{R}^d \\ \|\mathbf{w}_1\|, \dots, \|\mathbf{w}_s\| = 1}} \sum_{i=1}^n \left\| \mathbf{x}_i - \sum_{j=1}^s \langle \mathbf{x}_i, \mathbf{w}_j \rangle \mathbf{w}_j \right\|_2^2.$$

2 Solve

$$[\mathbb{W}^*, \mathbb{U}^*] = \arg \min_{\substack{\mathbb{W} \in \mathbb{R}^{s \times d} \\ \mathbb{U} \in \mathbb{R}^{d \times s}}} \sum_{i=1}^n \|\mathbf{x}_i - \mathbb{U} \mathbb{W} \mathbf{x}_i\|_2^2.$$

3 Find the s largest eigenvalue of the matrix $\mathbb{V}$.

In the sense that $\mathbb{U}^* = [\mathbf{w}_1^*, \dots, \mathbf{w}_s^*]$ and $\mathbb{W}^* = \mathbb{U}^{*T}$. Moreover, $\mathbf{w}_j^*$ is the eigenvector corresponds to the j th largest eigenvalue.

Exercise

Let

$$[\mathbf{w}_1^*, \dots, \mathbf{w}_s^*] := \arg \min_{\substack{\mathbf{w}_1, \dots, \mathbf{w}_s \in \mathbb{R}^d \\ \|\mathbf{w}_1\|, \dots, \|\mathbf{w}_s\| = 1}} \sum_{i=1}^n \left\| \mathbf{x}_i - \sum_{j=1}^s \langle \mathbf{x}_i, \mathbf{w}_j \rangle \mathbf{w}_j \right\|_2^2.$$

Calculate

$$\sum_{i=1}^n \left\| \mathbf{x}_i - \sum_{j=1}^s \langle \mathbf{x}_i, \mathbf{w}_j^* \rangle \mathbf{w}_j^* \right\|_2^2$$

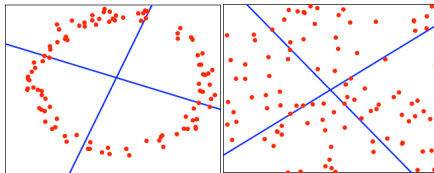
using the eigenvalues of $\mathbb{V}$. Assume that $\{\lambda_j\}_{j=1}^d$ is the eigenvalues ordered in a non-increasing manner.

Remark

- The eigenvectors of $\mathbb{V}$ are all orthogonal to each other, so together they span the whole d -dimensional space.
- The first principal component, i.e. the eigenvector which corresponds to the largest value of $\mathbb{V}$, is the direction along which the data have the largest variance.
- The second principal component, i.e. the second eigenvector, is the direction orthogonal to the first component with the largest variance. Because it is orthogonal to the first eigenvector, their projections will be uncorrelated.
- In fact, projections on to all the principal components are uncorrelated with each other. If we use s principal components, our weight matrix $\mathbb{W}$ will be a $d \times s$ matrix. The variance of the projections on to the first s principal components is then
$$\sum_{i=1}^s \lambda_i.$$

4. Kernel PCA

Motivation: Is linear low-dimensional manifold sufficient?



- PCA will make no difference between these examples, because the structure on the left is not linear
- Are there ways to find nonlinear, low-dimensional manifold?

How to make PCA non-linear?

- Let $S := \{\mathbf{x}_1, \dots, \mathbf{x}_n\} \subset \mathbb{R}^d$ be the original data with dimension $d \gg 1$ huge. Let $\mathcal{X}$ be some instance space such that $S \subset \mathcal{X}$. Consider a feature mapping: $\psi : \mathcal{X} \rightarrow \mathcal{H}$ with $\mathcal{H}$ being some Hilbert space (possibly of infinite dimension).
- Let $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ be a kernel function, i.e.,

$$K(\mathbf{x}, \mathbf{x}') = \langle \psi(\mathbf{x}), \psi(\mathbf{x}') \rangle_{\mathcal{H}}.$$

- Kernel PCA is the process of mapping the elements in S into $\mathcal{H}$ using the feature mapping ψ , and then applying PCA⁴ over

$$\{\psi(\mathbf{x}_1), \dots, \psi(\mathbf{x}_n)\}.$$

The output of this process is the set of reduced elements.

⁴Note that this PCA is nonlinear in the original data S

- Assume that the mean of the data in the feature space is zero⁵, i.e.,

$$\sum_{i=1}^n \psi(\mathbf{x}_i) = 0.$$

- Define the covariance matrix⁶,

$$\mathbb{V} = \frac{1}{n} \sum_{i=1}^n \psi(\mathbf{x}_i) \psi(\mathbf{x}_i)^T.$$

- The eigenvalue problem is to find the eigenpair $(\lambda_j, \mathbf{v}_j) \in \mathbb{R} \times \mathcal{H}$, satisfying⁷

$$\begin{cases} \mathbb{V} \mathbf{v}_j = \lambda_j \mathbf{v}_j, & j = 1, \dots \\ \mathbf{v}_j^T \mathbf{v}_j = 1. \end{cases}$$

⁵if not, then we should make the data first by subtracting the mean.

⁶Using the definition of the covariance for a random vector ξ with mean zero, we have $\text{Var}[\xi] = \mathbb{E}[\xi \xi^T]$.

⁷Here, we denote $\mathbf{v}^T \mathbf{v} := \langle \mathbf{v}, \mathbf{v} \rangle_{\mathcal{H}}$ for $\mathbf{v} \in \mathcal{H}$.

Note on the covariance matrix $\mathbb{C}$

$$\mathbb{V} = \frac{1}{n} \sum_{i=1}^n \psi(\mathbf{x}_i) \psi(\mathbf{x}_i)^T.$$

- we derive this covariance matrix under the **interpretation of PCA as variance maximization**, see slide 64 for more details.
- The main idea of interpretation of PCA as variance maximization is to introduce a **random vector ξ that follows uniform distribution over the sample space $\Omega := \{\psi(\mathbf{x}_1), \dots, \psi(\mathbf{x}_n)\}$** , and to **seek $\omega \in \mathcal{H}$ with unit norm, s.t., the random variable $\langle \omega, \xi \rangle_{\mathcal{H}}$ has maximal variance**. That is, we would like to solve the problem⁸

$$\arg \max_{\omega \in \mathcal{H}: \|\omega\|_{\mathcal{H}}=1} \text{Var}[\langle \omega, \xi \rangle_{\mathcal{H}}] = \arg \max_{\omega \in \mathcal{H}: \|\omega\|_{\mathcal{H}}=1} \frac{1}{n} \sum_{i=1}^n |\langle \omega, \psi(\mathbf{x}_i) \rangle_{\mathcal{H}}|^2$$

⁸You can apply the kernel trick to see that we only need to search for ω in a finite dimensional space spanned by $\{\psi(\mathbf{x}_1), \dots, \psi(\mathbf{x}_n)\}$.

- Then ω is the principal component.
- Assume that we have already found the first k principal component of the data $\{\psi(\mathbf{x}_1), \dots, \psi(\mathbf{x}_n)\}$. We seek the $k + 1$ th principal component by seeking $\omega \in \mathcal{H}$ with unit norm and orthogonal to the first k principal component, s.t., the random variable $\langle \omega, \xi \rangle_{\mathcal{H}}$ has maximal variance.

- We can avoid going to the feature space $\mathcal{H}$ by using the **Gram matrix** $\mathbb{K} \in \mathbb{R}^{n \times n}$,

$$\mathbb{K}_{i,k} := K(\mathbf{x}_i, \mathbf{x}_k) = \langle \psi(\mathbf{x}_i), \psi(\mathbf{x}_k) \rangle_{\mathcal{H}}.$$

- Note that each eigenfunction v_j can be written as a linear combination of the features,

$$v_j = \sum_{i=1}^n a_{ji} \psi(\mathbf{x}_i)$$

- Finding the eigenfunctions is equivalent to finding the coefficients a_{ji} for $j = 1, 2, \dots$.

- By substituting this into the eigenvalue problem, we obtain

$$\frac{1}{n} \sum_{i=1}^n \psi(\mathbf{x}_i) \psi(\mathbf{x}_i)^T \left(\sum_{i=1}^n a_{ji} \psi(\mathbf{x}_i) \right) = \lambda_j \left(\sum_{i=1}^n a_{ji} \psi(\mathbf{x}_i) \right), \quad j = 1, 2, \dots$$

- using the kernel, we derive,

$$\frac{1}{n} \sum_{i=1}^n \psi(\mathbf{x}_i) \left(\sum_{\ell=1}^n a_{j\ell} K(\mathbf{x}_i, \mathbf{x}_\ell) \right) = \lambda_j \left(\sum_{i=1}^n a_{ji} \psi(\mathbf{x}_i) \right), \quad j = 1, 2, \dots$$

- A small trick: multiply this by $\psi(\mathbf{x}_k)^T$ to the left, we obtain

$$\frac{1}{n} \sum_{i=1}^n \psi(\mathbf{x}_k)^T \psi(\mathbf{x}_i) \left(\sum_{\ell=1}^n a_{j\ell} K(\mathbf{x}_i, \mathbf{x}_\ell) \right) = \lambda_j \left(\sum_{i=1}^n a_{ji} \psi(\mathbf{x}_k)^T \psi(\mathbf{x}_i) \right),$$

- Finally, we obtain the matrix form,

$$\mathbb{K}^2 \mathbf{a}_j = n\lambda_j \mathbb{K} \mathbf{a}_j$$

- We can remove a factor of $\mathbb{K}$ from both sides⁹,

$$\mathbb{K} \mathbf{a}_j = n\lambda_j \mathbf{a}_j.$$

⁹This will only affect eigenfunctions with zero eigenvalue, which will not be principal components

- The normalization condition leads to

$$\mathbf{a}_j^T \mathbb{K} \mathbf{a}_j = 1.$$

- Plug it into the governing equation yields,

$$\lambda_j n \mathbf{a}_j^T \mathbf{a}_j = 1 \text{ for all } j.$$

- For a new point $\mathbf{x} \in \mathcal{X}$, its projection onto the principal components are

$$\psi(\mathbf{x})^T \mathbf{v}_j = \sum_{i=1}^n a_{ji} K(\mathbf{x}, \mathbf{x}_i).$$

Summary of kernel PCA

- 1 Pick a kernel
- 2 construction a normalized kernel matrix $\mathbb{K}$ of the data of dimension $n \times n$
- 3 Find the eigenpairs $(\lambda_j, \mathbf{a}_j)$ of this kernel matrix
- 4 For any data point, we can represent it as the following set of features,

$$y_j = \sum_{i=1}^n a_{ji} K(\mathbf{x}, \mathbf{x}_i), \quad j = 1, \dots, n$$

- 5 We can limit the number of components to $s \ll n$ for a more compact representation by picking the $\mathbf{a}$'s corresponding to the largest eigenvalues.

3.5 Applications of PCA

PCA has wide applications

- **Development indexes:** the City Development Index was developed by PCA from about 200 indicators of city outcomes. It ultimately used about 15 indicators but was a good predictor of many more variables. [Wiki link for more information on City Development Index]
- **Population genetics:** genetics varies largely according to proximity, so the first two principal components actually show spatial distribution.
[Novembre, John; Stephens, Matthew (2008), "Interpreting principal component analyses of spatial population genetic variation", Nat Genet.]
- **Market research and indexes of attitude:** PCA is used to develop customer satisfaction or customer loyalty scores for products.
[DeSarbo, Wayne; Hausmann, Robert; Kukitz, Jeffrey (2007). "Restricted principal components analysis for marketing research", Journal of Modelling in Management.]

1. Application: US Zero-Coupon Rates¹⁰

¹⁰Based on Chapter 9 of *Financial Data Analytics: with Machine Learning, Optimization and Statistics*, written by Chen, Cheung, and Yam.

Analysis of US Zero-Coupon Rates

Definition (Bond)

A bond is a portal through which a corporate or governmental body raises capital. When bonds are issued, investors purchase those bonds, effectively acting as lenders to the issuing entity. The investors earn a return in the form of coupon payments, which are made semiannually or annually, throughout the life of the bond.

Definition (Zero-Coupon Bond)

A zero-coupon bond, also known as an accrual bond, is a debt security that does not pay interest but instead trades at a deep discount, rendering a profit at maturity, when the bond is redeemed for its full face value.

The interest earned on a zero-coupon bond

Definition (Imputed Interest)

The interest earned on a zero-coupon bond is an imputed interest, meaning that it is an estimated interest rate for the bond and not an established interest rate.

Example

Consider a bond with a face amount of \$20,000, that matures in 20 years:

- With a 5.5% yield, it may be purchased for roughly \$6,855.

$$6855(1 + 5.5\%)^{20} \approx 20000$$

- At the end of the 20 years, the investor will receive \$20,000.
- The difference between \$20,000 and \$6,855 represents the interest that compounds automatically until the bond matures.

Raw data

Data File

Our data file “us-rate.csv” contains US semi-annualized zero-coupon rates, measured in basis points (b.p. for short, and $100\text{bps} = 1\%$), with maturities of 1M (months) to 15Y (years), monthly data over the period of 1944 to 1992.

How to read in the data file?

In Python, one can install Pandas, a powerful Python data analysis toolkit. The handbook can be found in the following link:
<https://pandas.pydata.org/docs/pandas.pdf>

Useful package

NumPy offers comprehensive mathematical functions, random number generators, linear algebra routines, which is a helpful tool in Python.

Observations on the data

Let's look at the correlation matrix of zero-coupon rates in Python.

```
1  [[1.    0.99 0.99 0.99 0.98 0.98 0.97 0.96 0.95 0.95 0.94 0.92 0.91]
2   [0.99 1.    1.    0.99 0.99 0.99 0.98 0.97 0.96 0.96 0.95 0.94 0.92]
3   [0.99 1.    1.    1.    1.    0.99 0.99 0.98 0.97 0.96 0.96 0.94 0.93]
4   [0.99 0.99 1.    1.    1.    1.    0.99 0.99 0.98 0.97 0.96 0.95 0.94]
5   [0.98 0.99 1.    1.    1.    1.    1.    0.99 0.98 0.98 0.97 0.96 0.94]
6   [0.98 0.99 0.99 1.    1.    1.    1.    1.    0.99 0.99 0.98 0.97 0.96]
7   [0.97 0.98 0.99 0.99 1.    1.    1.    1.    0.99 0.99 0.98 0.98 0.96]
8   [0.96 0.97 0.98 0.99 0.99 1.    1.    1.    1.    0.99 0.99 0.98]
9   [0.95 0.96 0.97 0.98 0.98 0.99 0.99 1.    1.    1.    0.99 0.98]
10  [0.95 0.96 0.96 0.97 0.98 0.99 0.99 1.    1.    1.    1.    0.99]
11  [0.94 0.95 0.96 0.96 0.97 0.98 0.98 0.99 1.    1.    1.    0.99]
12  [0.92 0.94 0.94 0.95 0.96 0.97 0.98 0.99 0.99 1.    1.    1.    ]
13  [0.91 0.92 0.93 0.94 0.94 0.96 0.96 0.98 0.98 0.99 0.99 1.    1.    ]]
```

Key Observation

The variables are highly correlated. This means that very likely we can use only a few variables to represent this dataset without losing a large amount of information. This is exactly the goal of dimension or variable reduction!

Which variables to use?

Question

Which variables to use so that they can contain the information in the dataset as much as possible?

Ans.

Let the original variables be $\mathbf{x}_1, \dots, \mathbf{x}_{13}$, then we look for a new variable

$$\mathbf{y}_1 = h_{1(1)}\mathbf{x}_1 + \dots + h_{1(13)}\mathbf{x}_{13} := \mathbb{X}\mathbf{h}_1$$

where $\mathbf{h}_1 = (h_{1(1)}, \dots, h_{1(13)})^\top$ and $\mathbb{X} = (\mathbf{x}_1, \dots, \mathbf{x}_{13})$ s.t. the variance of $\mathbf{y}_1$:

$$\text{Var}(\mathbf{y}_1) := \mathbf{h}_1^\top \mathbb{S} \mathbf{h}_1$$

is maximum subject to the constraint:

$$h_{1(1)}^2 + \dots + h_{1(13)}^2 = \langle \mathbf{h}_1, \mathbf{h}_1 \rangle = 1$$

where $\mathbb{S}$ is the sample covariance matrix of $\mathbf{x}_1, \dots, \mathbf{x}_{13}$. This is the first principal component (PC).

Which variables to use?

us-rate												
1m	3m	6m	9m	12m	18m	2y	3y	4y	5y	7y	10y	15y
0.34	0.51	0.7	0.79	0.84	0.9	0.93	1.24	1.4	1.65	1.93	2.16	2.33
0.29	0.36	0.73	0.79	0.82	0.9	0.94	1.25	1.41	1.64	1.9	2.12	2.3
0.32	0.42	0.78	0.8	0.81	0.88	0.92	1.25	1.42	1.65	1.91	2.11	2.27
0.29	0.4	0.7	0.76	0.78	0.89	0.94	1.23	1.37	1.63	1.92	2.13	2.28
0.3	0.55	0.74	0.78	0.79	0.89	0.94	1.24	1.39	1.64	1.92	2.12	2.28
0.32	0.52	0.8	0.78	0.76	0.9	0.96	1.26	1.41	1.65	1.93	2.14	2.3
0.29	0.53	0.7	0.74	0.76	0.87	0.93	1.24	1.39	1.64	1.92	2.13	2.3
0.29	0.39	0.69	0.73	0.75	0.89	0.95	1.24	1.38	1.62	1.9	2.1	2.26
0.33	0.43	0.78	0.78	0.78	0.9	0.96	1.25	1.4	1.64	1.91	2.12	2.27
0.3	0.45	0.78	0.77	0.77	0.94	1.02	1.29	1.42	1.66	1.93	2.13	2.29
...	...	...	...	...	...	...	...	...	...	...	...	...
x1												x13

Which variables to use?

Ans.

We can continue to find the second principal component such that

$$\mathbf{y}_2 = h_{2(1)}\mathbf{x}_1 + \dots + h_{2(13)}\mathbf{x}_{13} := \mathbb{X}\mathbf{h}_2$$

where $\mathbf{h}_2 = (h_{2(1)}, \dots, h_{2(13)})^\top$ such that the variance of $\mathbf{y}_2$:

$$\text{Var}(\mathbf{y}_2) = \mathbf{h}_2^\top \mathbb{S} \mathbf{h}_2$$

is maximum subject to

$$h_{2(1)}^2 + \dots + h_{2(13)}^2 := \langle \mathbf{h}_2, \mathbf{h}_2 \rangle = 1$$

and $\langle \mathbf{h}_1, \mathbf{h}_2 \rangle = 0$, which means that $\mathbf{h}_1$ and $\mathbf{h}_2$ are orthogonal to each other.

The third PC can be found similarly, and the process can continue up to the last 13—th PC. This applies the basic idea of PCA.

Remark

In the process of finding the second PC, we know $\mathbf{h}_1$ and $\mathbf{h}_2$ are orthogonal to each other. Equivalently, that means $\mathbf{y}_1$ and $\mathbf{y}_2$ are statistically uncorrelated. First note that

$$\text{Cov}(\mathbf{y}_i, \mathbf{y}_j) = \mathbf{h}_i^\top \mathbb{S} \mathbf{h}_j,$$

for $i, j = 1, 2, \dots, 13$; but each $\mathbf{h}_k$ is an eigenvector of $\mathbb{S}$ with respect to the eigenvalue λ_k , therefore

$$\text{Cov}(\mathbf{y}_i, \mathbf{y}_j) = \mathbf{h}_i^\top (\lambda_j \mathbf{h}_j) = \lambda_j \langle \mathbf{h}_i, \mathbf{h}_j \rangle = 0.$$

PCA algorithm in Python

Built-in Function

PCA is implemented in Python through the “PCA ()” function in “sklearn.decomposition”. First let us illustrate it by using the “us-rate.csv” file as an example.

Codes

```
1 from sklearn.decomposition import PCA
2
3 np.set_printoptions(precision=4)      # Display the number of 4 digits
4 pca = PCA()
5 standard_d = (d - np.mean(d, axis=0))/np.std(d, axis=0)
6 pca.fit(standard_d)
7 # components_: an ndarray of shape (n_components, n_features)
8 print(pca.components_[:6,:].T)
```

PCA algorithm in Python

Recall that the first PC is

$$\mathbf{y}_1 = h_{1(1)}\mathbf{x}_1 + \dots + h_{1(13)}\mathbf{x}_{13} = \mathbb{X}\mathbf{h}_1$$

where $\mathbb{X} = (\mathbf{x}_1, \dots, \mathbf{x}_{13})$ is a matrix containing the raw data and $\mathbf{h}_1 = (h_{11}, \dots, h_{113})$ is called the **loading** for the first PC. The first six PCs' loadings for the US rates example are

1	[	0.2732	0.4043	-0.5976	-0.5659	-0.2719	0.0604]
2	[	0.2758	0.3446	-0.2823	0.2941	0.6122	-0.3492]
3	[	0.277	0.3027	0.0285	0.3677	0.1699	0.3833]
4	[	0.2783	0.2361	0.1771	0.2332	-0.2162	0.2511]
5	[	0.2786	0.2023	0.252	0.1651	-0.4103	0.1819]
6	[	0.2798	0.0915	0.2765	-0.0345	-0.2009	-0.3118]
7	[	0.2799	0.0353	0.2885	-0.1355	-0.0939	-0.5615]
8	[	0.2798	-0.0998	0.2226	-0.2212	0.1388	-0.1092]
9	[	0.2791	-0.1682	0.1881	-0.2654	0.2558	0.1215]
10	[	0.2785	-0.2298	0.0836	-0.2061	0.2137	0.2002]
11	[	0.2772	-0.3015	-0.0391	-0.1377	0.1623	0.2945]
12	[	0.2754	-0.3754	-0.2187	0.1056	-0.0493	0.0824]
13	[	0.2729	-0.4438	-0.4082	0.4062	-0.3146	-0.242]]

Result analysis

- From the output, the first and second PCs are

$$\mathbf{y}_1 = 0.2732\mathbf{x}_1 + \cdots + 0.2729\mathbf{x}_{13}$$

$$\mathbf{y}_2 = 0.4043\mathbf{x}_1 + \cdots - 0.4438\mathbf{x}_{13}$$

respectively.

- It is easy to check that $\langle \mathbf{h}_1, \mathbf{h}_1 \rangle = \langle \mathbf{h}_2, \mathbf{h}_2 \rangle = 1$ (of unit length) and $\langle \mathbf{h}_1, \mathbf{h}_2 \rangle = 0$ (being orthogonal). Moreover, any of these PCs are of unit length and any pairs of these PCs are orthogonal.

Question

We can display all of these 13 PCs. However, PCA aims at **variable reduction**. We want to represent this dataset by using only a few variables instead of all 13 variables. How many PCs should we use?

How many PCs should we use?

Before answering this question, first let us look at the variance or standard deviation (s.d.) of these PCs. The following program is a justification on the respective unit norm nature and their orthogonality of the first two principal components from the US rates example in Python.

```
1 pc1 = pca.components_[0,:]      # Save the loading of 1st PC
2 pc2 = pca.components_[1,:]      # Save the loading of 2nd PC
3
4 print(pc1 @ pc1)                #  $h_1^\top h_1$ 
5 print(pc2 @ pc2)                #  $h_2^\top h_2$ 
6 print(pc1 @ pc2)                #  $h_1^\top h_2$ 
```

```
1 1.0
2 0.99999999999999989
3 1.3877787807814457e-17
```

How many PCs should we use?

The variances of different principal components of the US rates example in Python are in the following

```
1 [[ 0.2732  0.4043 -0.5976 -0.5659 -0.2719  0.0604]
2  [ 0.2758  0.3446 -0.2823  0.2941  0.6122 -0.3492]
3  [ 0.277   0.3027  0.0285  0.3677  0.1699  0.3833]
4  [ 0.2783  0.2361  0.1771  0.2332 -0.2162  0.2511]
5  [ 0.2786  0.2023  0.252   0.1651 -0.4103  0.1819]
6  [ 0.2798  0.0915  0.2765 -0.0345 -0.2009 -0.3118]
7  [ 0.2799  0.0353  0.2885 -0.1355 -0.0939 -0.5615]
8  [ 0.2798 -0.0998  0.2226 -0.2212  0.1388 -0.1092]
9  [ 0.2791 -0.1682  0.1881 -0.2654  0.2558  0.1215]
10 [ 0.2785 -0.2298  0.0836 -0.2061  0.2137  0.2002]
11 [ 0.2772 -0.3015 -0.0391 -0.1377  0.1623  0.2945]
12 [ 0.2754 -0.3754 -0.2187  0.1056 -0.0493  0.0824]
13 [ 0.2729 -0.4438 -0.4082  0.4062 -0.3146 -0.242 ]]
```

result

The first PC explained $12.7239/13 = 97.88\%$ of the total variance of 13. The second PC explained 1.83%, and so on. If we only use the first PC, this PC already preserved almost all (97.88%) the information. If we use the first two PCs, we can preserve 99.71% of all the information, and so on.

How much information we lose by using the first few PCs?

- From the general theory of PCA, all y_i 's are orthogonal to each other; indeed, for $\mathbb{H} = [\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_{13}]$, $\mathbb{S} = \mathbb{H}\mathbb{D}\mathbb{H}^\top$ and $\mathbf{y} = \mathbb{H}^\top \mathbf{x}$,

$$\text{Cov}(\mathbf{y}) = \text{Cov}(\mathbb{H}^\top \mathbf{x}) = \mathbb{H}^\top \text{Cov}(\mathbf{x}) \mathbb{H} = \mathbb{H}^\top \mathbb{H} \mathbb{D} \mathbb{H}^\top \mathbb{H} = \mathbb{D}$$

which is a diagonal matrix of eigenvalues, and so all off-diagonal elements vanished, i.e. $\text{Cov}(\mathbf{y}_i, \mathbf{y}_j) = 0$, for $i \neq j$.

- Define $\mathbf{y}_i = \sum_{j=1}^{13} h_{ij} \mathbf{x}_j$, for $i = 1, 2, \dots, 13$, or equivalently $\mathbf{y} = \mathbb{A} \mathbf{x}$,

where $\mathbb{A} = (h_{ij})_{13 \times 13}$ and hence $\mathbb{A} = \mathbb{H}^\top$. Now, $\mathbf{x} = \mathbb{H} \mathbf{y}$, i.e.

$$\mathbf{x}_i = \sum_{j=1}^{13} h_{ij} \mathbf{y}_j, \text{ and clearly } \sum_{j=1}^{13} h_{ij}^2 = 1 \text{ with } h_{ij}^2 \leq 1 \text{ for all } j = 1, \dots, 13.$$

How much information we lose by using the first few PCs?

- Then we have the residual information, of the total variance, by using only the first m PCs, for $i = 1, 2, \dots, 13$:

$$\begin{aligned}\text{Var} \left(\mathbf{x}_i - \sum_{j=1}^m h_{ij} \mathbf{y}_j \right) &= \text{Var} \left(\sum_{j=m+1}^{13} h_{ij} \mathbf{y}_j \right) \\ &= \sum_{j=m+1}^{13} h_{ij}^2 \text{Var}(\mathbf{y}_j) \\ &= \sum_{j=m+1}^{13} h_{ij}^2 \lambda_j \\ &\leq \sum_{j=m+1}^p \lambda_j\end{aligned}$$

How much information we lose by using the first few PCs?

- For the US rates example, if we choose $m = 3$, the latter sum is even less than 0.01% of the total variance for each $i = 1, 2, \dots, 13$.
- Therefore, the **total L^2 -error, by using only the first m PCs**, is

$$\begin{aligned}\sum_{i=1}^{13} \text{Var} \left(\mathbf{x}_i - \sum_{j=1}^m h_{ij} \mathbf{y}_j \right) &= \sum_{i=1}^{13} \sum_{j=m+1}^{13} h_{ij}^2 \lambda_j = \sum_{j=m+1}^{13} \lambda_j \sum_{i=1}^{13} h_{ij}^2 \\ &= \sum_{j=m+1}^{13} \lambda_j \sum_{i=1}^{13} h_{ji}^2 = \sum_{j=m+1}^{13} \lambda_j\end{aligned}$$

since $\mathbb{A} = \mathbb{H}^\top$ is orthogonal, i.e. $\langle \mathbf{h}_j, \mathbf{h}_j \rangle = 1$.

- Again, if we set $m = 3$, there is only less than 0.01% loss of the total variance in our example.

Financial interpretation of PCs obtained

Moral

Many investment instruments are very sensitive to the movement of interest rates, such as stocks, bonds. Knowledge on these components is very useful in scientifically valuating these instruments.

The first PC

Recall that the first PC is

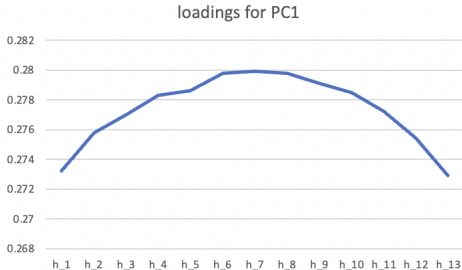
$$\begin{aligned} \mathbf{y}_1 = & 0.2732\mathbf{x}_1 + 0.2758\mathbf{x}_2 + 0.277\mathbf{x}_3 + 0.2783\mathbf{x}_4 + 0.2786\mathbf{x}_5 \\ & + 0.2798\mathbf{x}_6 + 0.2799\mathbf{x}_7 + 0.2798\mathbf{x}_8 + 0.2791\mathbf{x}_9 + 0.2785\mathbf{x}_{10} \\ & + 0.2772\mathbf{x}_{11} + 0.2754\mathbf{x}_{12} + 0.2729\mathbf{x}_{13}. \end{aligned}$$

The loadings are almost constant at the level of 0.27 for bonds of all maturity times.

In fact, these loadings are unique only up to a positive or negative sign. That is, if we define a new first PC as

$$\begin{aligned} \mathbf{y}'_1 = -\mathbf{y}_1 = & -0.2732\mathbf{x}_1 - 0.2758\mathbf{x}_2 - 0.277\mathbf{x}_3 - 0.2783\mathbf{x}_4 - 0.2786\mathbf{x}_5 \\ & - 0.2798\mathbf{x}_6 - 0.2799\mathbf{x}_7 - 0.2798\mathbf{x}_8 - 0.2791\mathbf{x}_9 - 0.2785\mathbf{x}_{10} \\ & - 0.2772\mathbf{x}_{11} - 0.2754\mathbf{x}_{12} - 0.2729\mathbf{x}_{13}. \end{aligned}$$

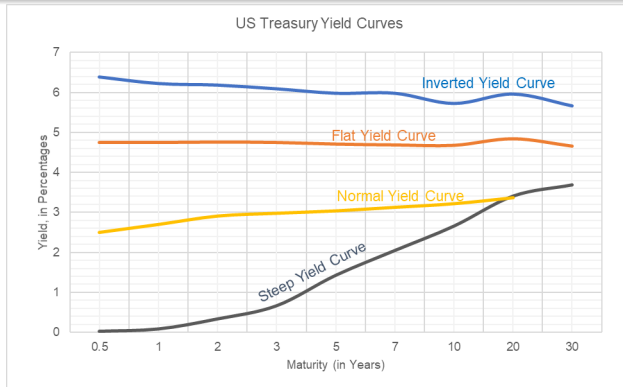
this alternative PC has the same variance as the old one.



Yield curve

Definition (Yield Curve)

The “yield curve” refers to the relationships among several factors: short-term interest rates, intermediate interest rates, and long-term interest rates. The shape of the curve of these points on a graph depends on the rates and maturities of the debt securities that are being measured.



Parallel shift

Definition (Parallel Shift)

A parallel shift in the yield curve happens when the interest rates on all fixed-income maturities increase or decrease by the same number of basis points. Such a change would shift the yield curve parallel to its present place on the graph without changing its slope.

When a parallel shift happens?

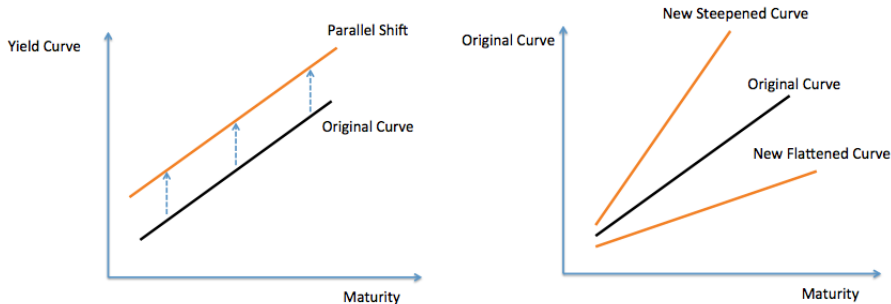
To explain this parallel shift, we refer to a shift in the economic environment caused by some major macroeconomic event, so that the rates of returns for all zero-coupon bonds with different maturities react in the same direction, and even move by essentially same amount of basis points.

Parallel shift

Example

If a 3M treasury bill increases 100bps, then all 6M, 1Y, 5Y, 10Y and 15Y rates will increase by a similar amount as well; also see more information in NASDAQ (<http://www.nasdaq.com>).

The following are two examples of parallel shift and non-parallel shift.



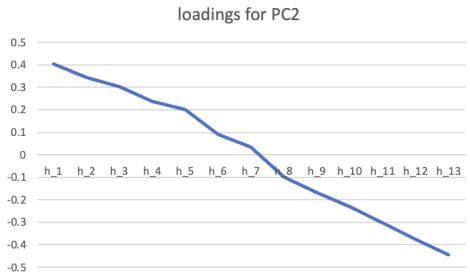
Financial interpretation of PCs obtained

The second PC

The second PC is

$$\begin{aligned} y_2 = & 0.4043x_1 + 0.3446x_2 + 0.3027x_3 + 0.2361x_4 + 0.2023x_5 \\ & + 0.0915x_6 + 0.0353x_7 - 0.0998x_8 - 0.1682x_9 - 0.2298x_{10} \\ & - 0.3015x_{11} - 0.3754x_{12} - 0.4438x_{13} \end{aligned}$$

where the loadings are decreasing with maturity.



Financial interpretation of PCs obtained

The second PC

Again, the loadings are not unique. The alternative second PC could be

$$\begin{aligned} y_2' = -y_2 = & -0.4043x_1 - 0.3446x_2 - 0.3027x_3 - 0.2361x_4 - 0.2023x_5 \\ & - 0.0915x_6 - 0.0353x_7 + 0.0998x_8 + 0.1682x_9 + 0.2298x_{10} \\ & + 0.3015x_{11} + 0.3754x_{12} + 0.4438x_{13} \end{aligned}$$

which seems more reasonable since it agrees with the liquidity preference theory.

Theorem (Liquidity Preference Theory)

Investors prefer to preserve their liquidity and invest in investment vehicles that last for a short period of time, and so additional premiums are paid for holding securities with longer time of maturity.

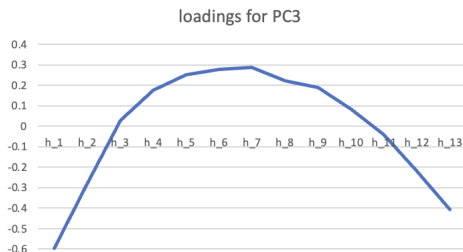
Financial interpretation of PCs obtained

The third PC

The third PC is

$$\begin{aligned} y_3 = & -0.5976x_1 - 0.2823x_2 + 0.0285x_3 + 0.1771x_4 + 0.252x_5 \\ & + 0.2765x_6 + 0.2885x_7 + 0.2226x_8 + 0.1881x_9 + 0.0836x_{10} \\ & - 0.0391x_{11} - 0.2187x_{12} - 0.4082x_{13} \end{aligned}$$

The loadings increase with maturity up to a certain point and then start decreasing again with maturity.



Financial interpretation of PCs obtained

Curvature

By considering the negative counterpart of y_3 , this effect is mainly caused by the higher demand on the very short-term and very long-term bonds; indeed, the borrowing market with short term bonds is popular with frequent traders in securities and derivatives markets; while for bonds of longer terms, they are welcomed by long term market participants, such as insurance and reinsurance companies, short term and long term bonds can be used for immunization of interest rate risk for investment vehicles of intermediate lifespans.

Principal component scores

Principal component scores

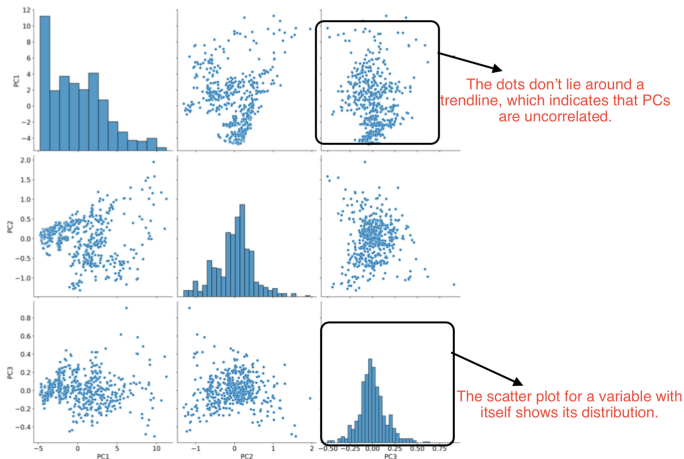
The original data can be expressed approximately in terms of $\mathbf{y}_1$, $\mathbf{y}_2$ and $\mathbf{y}_3$ by using the loadings of PC1, PC2 and PC3 in light of the fact that $\mathbf{y} = \mathbf{A}\mathbf{x} = \mathbf{H}^T \mathbf{x}$. These $\mathbf{y}_i$'s are also called the principal component scores with respect to PC1, PC2 and PC3, respectively. The geometric interpretation of PCA may be illustrated by the scatter plot matrix of these scores.

The following program shows how to do the scatter plot for the first three principal components in Python.

```
1 from seaborn import pairplot
2
3 score = standard_d @ pca.components_.T # save scores of all PC's
4 score.columns = [f"PC{i}" for i in PC_comp]
5 print(score.iloc[:, :3])
6 plots = pairplot(score.iloc[:, :3], diag_kind="hist", height=5)
7 plots.savefig("PCA first 3 scores pairplot.png", dpi=400)
```

Principal component scores

A scatter plot uses dots to represent values for two different numeric variables. Here shows the scatter plot for the first three principal components from the US rates example. These components are expected to be uncorrelated.



Financial interpretation of PCs obtained

Summary

- Each vector of loadings is the direction that the corresponding sample variance of the transformed variables is maximized.
- Recall that the loadings of i -th PC are listed as

$$\mathbf{y}_i = h_{i(1)}\mathbf{x}_1 + \cdots + h_{i(13)}\mathbf{x}_{13} = \mathbb{X}\mathbf{h}_i$$

where $\mathbf{h}_i$ is the i -th unit eigenvector of $\mathbb{S}$ corresponding to the i -th largest eigenvalue.

- Form the $p \times p$ matrix $\mathbb{H} = (h_{ij})_{13 \times 13} = \left(\mathbf{h}_1^\top, \mathbf{h}_2^\top, \dots, \mathbf{h}_{13}^\top \right)$ such that

$$\mathbb{H}^\top = \mathbb{A}$$

this orthogonal matrix $\mathbb{H}$ diagonalizes the covariance matrix $\mathbb{S}$, and $\mathbb{H}$ is orthogonal, i.e., $\mathbb{H}^\top \mathbb{H} = \mathbb{H} \mathbb{H}^\top = \mathbb{I}_{13}$, the 13×13 identity matrix.

Summary

- Furthermore, $\mathbb{Y} = \mathbb{H}^\top \mathbb{X}$, a 1×13 vector with the first element being the PC1 score of x , and so on. The original datum x can be expressed in terms of y as $\mathbb{X} = \mathbb{H}\mathbb{Y}$, i.e., $\mathbf{x}_i = h_{i(1)}\mathbf{y}_1 + \cdots + h_{i(13)}\mathbf{y}_{13}$.
- In practice, we may only use the first few m PCs to approximate x , $\mathbf{x}_i \approx h_{i(1)}\mathbf{y}_1 + \cdots + h_{i(m)}\mathbf{y}_m$.
- This provides an interesting and useful application of PCA to approximate the Value-at-Risk of a portfolio and immunizes the portfolio from various risk factors arising in markets.

2. Application: Netflix recommendation system (self-study) ¹¹

¹¹based on [Kozma, Ilin, Raiko (2009, September). Binary principal component analysis in the Netflix collaborative filtering task. IEEE.]

PCA applied in Netflix recommendation system

Introduction

Recommendation systems form an integral component of many modern e-commerce websites. Users often rate items on a numerical scale (e.g., from 1 to 5). If the system is capable of accurately predicting the rating a user would give to an item, it can use this knowledge to make recommendations to the user.

- Dataset: the collected ratings can be represented in the form of a matrix in which each column contains ratings given by one user and each row contains ratings given to one item.
- Goal: naturally most of the entries in the matrix are missing (as any given user rates just a subset of the items) and the goal is to predict these missing values.

Binarization

As a preprocessing step, the ratings are encoded with binary values, according to the following scheme:

1 $\rightarrow$ 0000

2 $\rightarrow$ 0001

3 $\rightarrow$ 0011

4 $\rightarrow$ 0111

5 $\rightarrow$ 1111

With this scheme, each element in the data tells whether a rating is greater or smaller than a particular threshold. This kind of binarization can be used also for continuous valued data.

Binary PCA

Binarized data

Denote the sparsely populated matrix of ratings from 1 to 5 stars by X . X has dimensions $m \times p$, where m and p are the number of movies and the number of people, respectively. After binarization, we obtain binary matrices X_1, X_2, X_3, X_4 (one for each digit of the rating values), which we summarize in one matrix Y with dimensions $d \times p$, where $d = 4m$.

Model

We model the probability of each element y_{ij} of matrix Y to be 1 using the following formula:

$$P(y_{ij} = 1) = \sigma(a_i^T s_j)$$

where a_i and s_j are column vectors with c elements and $\sigma(\cdot)$ is the logistic function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

PCA process

The features of the movies and people are not known and have to be estimated to fit the available ratings. We summarize the features in two matrices

$$A = \begin{bmatrix} a_1 & \dots & a_d \end{bmatrix}^T$$
$$S = \begin{bmatrix} s_1 & \dots & s_p \end{bmatrix}$$

We express the likelihood of Y as a product of Bernoulli distributions

$$L = \prod_{ij \in O} \sigma(a_i^T s_j)^{y_{ij}} \left(1 - \sigma(a_i^T s_j)\right)^{(1-y_{ij})},$$

where O denotes the set of elements in Y corresponding to observed ratings. PCA is applied to estimate A and S (one can check the original paper for more details).

Prediction

Once A and S have been estimated, one can compute the probabilities for each element of matrices X_l . We can then transform the probabilities to the actual ratings by first calculating the probability of each particular rating value:

$$P_{x=1} = (1 - x_1)(1 - x_2)(1 - x_3)(1 - x_4)$$

$$P_{x=2} = (1 - x_1)(1 - x_2)(1 - x_3)x_4$$

$$P_{x=3} = (1 - x_1)(1 - x_2)x_3x_4$$

$$P_{x=4} = (1 - x_1)x_2x_3x_4$$

$$P_{x=5} = x_1x_2x_3x_4$$

where x is one rating in matrix X , $P_{x=r}$ denotes the probability that $x = r$ and $x_l (l = 1, \dots, 4)$ are the probabilities computed for the corresponding elements in matrices X_l . Then we estimate the rating as expectation

$$\hat{x} = \frac{1P_{x=1} + 2P_{x=2} + 3P_{x=3} + 4P_{x=4} + 5P_{x=5}}{P_{x=1} + P_{x=2} + P_{x=3} + P_{x=4} + P_{x=5}}.$$